

**SIMATS ENGINEERING  
ASSESSMENT TOOL 2**

**COURSE CODE: CSA5112    COURSE NAME: Cryptography and Network Security**

---

**COURSE OUTCOME COVERED**

**CO3:** *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

---

**QUESTIONS: 5**

**TOTAL MARKS: 25**

**Annexure A  
CASE STUDY**

---

***Code of Conduct:***

*I **T.Gokulnath(192521139)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

***T.Gokulnath***

## Annexure A

### 1. Weak Password Hashing in Web Application

**A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.**

**Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability.**

**Recommend a secure password hashing strategy and justify your choice.**

***Scenario:** A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.*

#### Analysis of the Vulnerability

**Speed of SHA-1:** SHA-1 is a general-purpose cryptographic hash function designed to be computed very quickly. This is a desirable property for data integrity checks, but it is exactly the wrong property for password storage, because it allows an attacker with commodity hardware (or GPUs/ASICs) to attempt billions of guesses per second in an offline brute-force or dictionary attack once the hash database is stolen.

**Absence of Salting:** A salt is a unique, random value appended to each password before hashing. Without a salt, two users who choose the same password produce identical hash values. This has two consequences: (1) an attacker only needs to crack one instance of a common password to compromise every account that shares it, and (2) the attacker can pre-compute a rainbow table — a large lookup table mapping plaintext candidates to their hash outputs — once, and reuse it against the entire stolen database instead of attacking each password individually.

**Combined Effect:** Because SHA-1 is fast and no salt was used, the breach allowed a single precomputed table (built once, offline, ahead of time) to instantly reverse a large fraction of the stored hashes, rather than forcing the attacker to mount a fresh brute-force search per account. This converts what should be a per-account, computationally expensive attack into a one-time, cheap, database-wide compromise.

**Cryptographic weaknesses of SHA-1 itself:** Independent of the salting issue, SHA-1 is also considered cryptographically broken for collision resistance (e.g., the 2017 "SHAttered" attack) and is deprecated by NIST for security-sensitive uses, making it an unsuitable primitive for any authentication-related purpose.

#### Recommended Secure Password Hashing Strategy

A modern password storage scheme should use a purpose-built, memory-hard, adaptive hashing algorithm rather than a general-purpose message digest. Recommended options, in order of preference:

- Argon2id (winner of the Password Hashing Competition) — memory-hard, tunable time/memory/parallelism cost, resistant to GPU/ASIC cracking and side-channel timing attacks.
- bcrypt — mature, widely deployed, adjustable work factor (cost parameter), automatically handles per-password salting.
- PBKDF2-HMAC-SHA256 — acceptable when FIPS compliance is required, with a high iteration count ( $\geq 310,000$  per current OWASP guidance) and a unique per-user salt.

Concrete recommendation: adopt Argon2id with a unique, cryptographically random salt ( $\geq 16$  bytes) generated per password, combined with a per-application secret "pepper" stored outside the database (e.g., in a hardware security module or secrets manager), and tuned cost parameters (memory, iterations, parallelism) so that hashing takes roughly 250–500 ms on production hardware.

### Justification

**Salting:** eliminates precomputed rainbow-table attacks and ensures identical passwords produce different stored hashes, so compromising one account reveals nothing about others.

**Memory-hardness / adaptive cost:** Argon2id and bcrypt deliberately consume significant CPU time and memory per hash attempt, which sharply reduces the number of guesses per second an attacker can perform even with specialized hardware — directly countering the brute-force speed advantage that made SHA-1 dangerous.

**Tunability:** The work factor can be increased over time as hardware becomes faster, preserving security margins without changing the algorithm.

**Defense in depth:** Adding a server-side pepper means that even a full database breach (hashes + salts) is insufficient to crack passwords without also compromising the separately stored pepper.

## 2. Digital Signature Compromise in Financial System

**A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.**

**Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.**

***Scenario:** A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.*

### Impact on Authentication and Non-Repudiation

**Authentication broken:** Digital signature schemes bind an identity to a message through possession of a private key: the recipient (or the system) verifies the signature using the corresponding public key and trusts that only the legitimate key holder could have produced it. Once the private key is stolen, this binding is broken — the attacker can produce signatures that are cryptographically indistinguishable from genuine ones, so the system can no longer reliably authenticate that the true account owner authorized the transaction.

**Non-repudiation undermined:** Non-repudiation relies on the assumption that only the legitimate owner has access to the private key, so a valid signature is treated as undeniable proof of intent. With the key compromised, this assumption fails: the genuine user could later (truthfully) deny performing the transaction, and the institution has no cryptographic way to distinguish attacker-signed transactions from user-signed ones without additional evidence. This creates disputes, potential financial liability, and erodes trust in the digital signature scheme as a legal/audit mechanism.

**Downstream consequences:** Every prior transaction signed with the same key also becomes suspect (retroactive doubt), the institution may face regulatory and reputational damage, and the

compromised key must be revoked and all counter-parties/systems informed, which itself takes time during which further fraud can occur.

## Mitigation Strategies

### Preventing private key compromise

- Store private keys in a Hardware Security Module (HSM) or a smart card / secure element so the raw key material never exists in exportable form on a general-purpose computer.
- Enforce multi-factor authentication before any signing operation is permitted (something the user has — token/HSM — plus something they know/are).
- Use short-lived session keys or transaction-specific signing keys derived from a securely stored master key, limiting the blast radius of any single compromise.
- Apply the principle of least privilege and network segmentation so malware on a user's workstation cannot directly access key-signing services.

### Detecting and responding to compromise

- Implement anomaly detection / behavioral analytics on transaction patterns (amount, frequency, geolocation, device fingerprint) to flag suspicious signed transactions for step-up verification.
- Maintain a Certificate Revocation List (CRL) or use OCSP so a compromised key's certificate can be revoked immediately and rejected by all verifiers.
- Require multi-signature / threshold-signature schemes for high-value transactions, so no single compromised key is sufficient to authorize a transfer.
- Log and timestamp all signing events immutably (e.g., append-only audit log or blockchain-anchored log) to support forensic investigation and limit repudiation disputes.

Together, hardware-backed key custody, multi-factor/multi-signature authorization, and rapid revocation restore the trust assumptions that authentication and non-repudiation depend on.

## 3. Kerberos Authentication Failure in Distributed System

**A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.**

**Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.**

***Scenario:** A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.*

### Impact of Time Synchronization on Kerberos

Kerberos authentication is fundamentally time-dependent. Tickets (Ticket Granting Tickets and Service Tickets) issued by the Key Distribution Center (KDC) carry timestamps and a limited validity window (lifetime), and the protocol relies on encrypted timestamps in the Authenticator to defend against replay attacks.

**Clock skew tolerance:** Kerberos implementations enforce a maximum allowable clock skew (commonly 5 minutes by default) between the client, server, and KDC. If a client's or server's clock drifts beyond this tolerance, the KDC or target service will reject otherwise valid tickets/authenticators as expired or "not yet valid".

**Replay protection depends on accurate clocks:** The Authenticator's timestamp is what allows a service to detect and reject replayed authentication requests within a short validity window. If clocks are unsynchronized, legitimate requests can be misclassified as replays (denial of service) or, in the worst case, a genuinely stale/replayed authenticator could be accepted if clocks are skewed in the attacker's favor, weakening replay protection.

**Ticket lifetime miscalculation:** Since ticket validity is calculated from absolute timestamps, a server whose clock is ahead or behind the KDC will compute an incorrect remaining lifetime, causing tickets to appear prematurely expired or, conversely, to be accepted after they should have expired.

**Operational impact:** The described symptom — frequent authentication failures — is the classic signature of clock drift across the distributed nodes rather than a credentials or configuration problem, and it will worsen as more nodes are added or as network partitions cause individual hosts to drift independently.

### Recommended Solutions

- Deploy a centralized, authoritative Network Time Protocol (NTP) or Precision Time Protocol (PTP) infrastructure that all Kerberos realm members (clients, application servers, and the KDC) synchronize against.
- Use a hierarchical NTP design with redundant, geographically distributed reference time sources to avoid a single point of failure for time synchronization.
- Monitor clock drift continuously (alerting when skew approaches the configured tolerance) so drifting hosts are corrected before they cause authentication failures.
- Configure the Kerberos realm's clock-skew tolerance (e.g., `krb5.conf` `clockskew` parameter) deliberately, balancing security (a tight window limits replay risk) against operational resilience (a slightly larger window tolerates minor, expected drift).
- Secure the time-sync protocol itself (authenticated NTP/NTS) since an attacker able to manipulate a host's clock could otherwise exploit the skew tolerance to extend ticket validity or facilitate replay attacks.
- Automate periodic time-sync health checks as part of infrastructure monitoring/alerting (e.g., via configuration management or observability tooling) so drift is remediated proactively rather than reactively.

With disciplined, authenticated time synchronization across all realm members, ticket validity checks and replay-detection windows function correctly, restoring reliable authentication.

## 4. Certificate Authority Breach in PKI System

**A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.**

**Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.**

***Scenario:** A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.*

### Impact on System Trust and Secure Communication

**Breakdown of the trust chain:** Public Key Infrastructure (PKI) rests on the assumption that a CA only signs certificates after properly verifying the requester's identity/domain control. A compromised CA can issue certificates for domains it does not control, and because the CA's root/intermediate certificate is already trusted by browsers and operating systems, these fraudulent certificates will be accepted as legitimate by any relying party — completely undermining the chain of trust.

**Man-in-the-middle attacks:** Fake certificates for trusted domains (e.g., a bank or email provider) allow an attacker to impersonate that service, intercept or modify TLS-protected traffic, and harvest credentials or sensitive data, all while the victim's browser shows a valid, trusted-looking connection.

**Erosion of confidentiality, integrity, and authentication guarantees:** The core security properties that TLS/X.509 are meant to provide — server authentication, confidentiality, and integrity of the session — are all defeated, since the attacker can present a certificate that passes normal validation.

**Wider ecosystem damage:** Because CAs are trusted transitively by many organizations and products, a single CA compromise can affect an enormous number of unrelated domains and users, and it damages confidence in the broader PKI trust model until remediated.

## Detection and Mitigation Mechanisms

### Detection

- Certificate Transparency (CT) logs — require/monitor publicly auditable, append-only logs of all issued certificates so fraudulently issued certificates for a domain become visible and can be spotted quickly by domain owners or monitoring services.
- Automated CT monitoring services / alerts that notify domain owners the moment an unexpected certificate is logged for their domain.
- Regular CA audits (e.g., WebTrust/ETSI audits) and Certificate Authority Authorization (CAA) DNS records that restrict which CAs are permitted to issue for a given domain, making unauthorized issuance easier to flag as a policy violation.

### Mitigation / Response

- Immediate revocation of the fraudulent certificates via CRLs and OCSP, combined with OCSP stapling and short-lived OCSP responses so revocation propagates quickly.
- Revoke and re-key the compromised CA's intermediate/root certificate if the compromise is at the signing-key level, and re-issue legitimate certificates under a trusted key.
- Adopt Certificate Pinning or HSTS-style mechanisms for critical applications so clients reject unexpected certificate chains even if nominally "trusted."
- Move toward shorter certificate lifetimes and automated reissuance (e.g., ACME/Let's Encrypt-style workflows) to reduce the window of exposure from any single compromised certificate.
- Harden the CA's key-management practices going forward — HSM-protected root keys kept offline, strict multi-person control (M-of-N) for signing operations, and network isolation of the issuing infrastructure — to prevent recurrence.

Together, Certificate Transparency for detection and rapid revocation/pinning for containment restore trust in the PKI ecosystem after a CA compromise.

## 5. Integrity Verification in Cloud Data Storage

**A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.**

**Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.**

***Scenario:** A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.*

### Analysis: How Incorrect Hashing Implementation Causes Integrity Failures

**Storing the hash alongside unprotected data:** If the integrity hash is stored in the same, equally accessible location as the file itself (e.g., in the same database record or bucket, without access controls or a trust anchor), an attacker who can modify the file can simply recompute the hash and overwrite the stored value too — defeating the entire purpose of the check.

**No authentication of the hash (using a plain hash instead of an HMAC/signature):** A bare hash (e.g., SHA-256 of the file) only proves that the file matches whatever value is currently stored as "the hash" — it does not prove that value was set by a legitimate party. Without a keyed MAC or digital signature over the hash, an attacker who can tamper with the data can also tamper with the reference hash, so the check silently passes.

**Weak or broken hash algorithms:** Use of deprecated algorithms (MD5, SHA-1) that are vulnerable to collision attacks could allow a malicious actor to craft a tampered file that produces the same hash as the original, passing validation despite the content being altered.

**Validation not actually enforced or checked at the right points:** The investigation describes "improper hash validation practices" — common failure modes include validating only at upload time and never again, failing open (treating a mismatch as a warning rather than blocking access), or not validating on every read/download, all of which let tampering that occurs after initial storage go undetected.

**No chain of custody / versioning:** Without logging who changed what and when, even a detected mismatch cannot be traced back to when and how the tampering occurred, delaying response.

### Recommended Robust Integrity Verification Mechanism

- Use a strong, collision-resistant hash function (SHA-256 or SHA-3) as the baseline digest algorithm rather than MD5/SHA-1.
- Protect the integrity value itself with an HMAC (using a secret key held separately from the storage system, e.g., in an HSM/KMS) or a digital signature, so the reference value cannot be forged even if an attacker can modify the stored file.
- Store the HMAC/signature and any versioning metadata in a separate, access-controlled, tamper-evident store (or use a Merkle-tree structure across file blocks, as used in systems like IPFS/blockchain-anchored storage) rather than co-located with the raw data.
- Validate integrity at every access point — on upload, on each download/read, and periodically via background integrity scans — and fail closed (block/alert) rather than fail open on a mismatch.
- Maintain immutable audit logs of hash computations and validation results, and consider anchoring periodic root hashes (Merkle roots) to an external, tamper-evident ledger for strong tamper-evidence over time.



## Justification of Effectiveness

**Keyed integrity (HMAC/digital signature):** ensures that only a party holding the secret key or private signing key can produce a valid integrity value, so an attacker who alters the file cannot also forge a matching, trusted integrity tag — closing the exact gap that caused the reported tampering incidents.

**Strong hash algorithms:** eliminate the feasibility of collision-based forgery.

**Separated, access-controlled storage of integrity metadata plus continuous validation and fail-closed behavior:** ensures tampering is detected at the earliest point of access rather than going unnoticed, restoring genuine, verifiable data integrity guarantees for cloud storage users.



**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>         | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                       | <b>Level 3 – Proficient</b>                    | <b>Level 2 – Developing</b>              | <b>Level 1 – Beginning</b>                            |
|-------------------------|------------------|------------------------------------------------------------------|------------------------------------------------|------------------------------------------|-------------------------------------------------------|
| Understanding of Case   | 25               | Thorough understanding ; clearly identifies all key issues       | Good understanding with most issues identified | Basic understanding ; some issues missed | Poor understanding ; problem not identified correctly |
| Application of Concepts | 25               | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps   | Limited application of concepts          | Fails to apply relevant concepts                      |
| Analysis                | 20               | In-depth analysis with strong reasoning and insights             | Good analysis with logical reasoning           | Basic analysis with limited depth        | Weak or no analysis; lacks reasoning                  |
| Solution Design         | 20               | Innovative, feasible solutions with strong justification         | Logical solutions with adequate justification  | Basic solutions with weak justification  | Incorrect or no solution provided                     |
| Clarity                 | 10               | Clear, well-structured, and easy to understand                   | Organized and understandable presentation      | Limited clarity and structure            | Poor clarity; difficult to understand                 |